

Faculdade de Computação e Informática – FCI

Curso: Ciência da Computação

Laboratório de Computação Paralela - Turma 05N-11

Prof. Mario Menezes

**Laboratório de Hierarquia de Memória,
Otimizações de Cache e Pthreads**

Localidade Espacial, Temporal, Tiling (Blocagem) e
Paralelismo em Memória Compartilhada

Nome: Gustavo Henrique de Sousa Santos

RA: 10721355

[Clique aqui: Link para o repositório GitHub](#)

Data de Submissão: 27/09/2026

Sumário

1. Introdução e Fundamentos Teóricos.....	3
2. Metodologia e Caracterização do Hardware.....	3
2.1. Caracterização da CPU e Memória RAM:.....	4
2.2. Topologia da Hierarquia de Cache:.....	4
2.3. Ambiente de Software e Compilação:.....	4
2.4. Ferramentas de Profiling:.....	4
3. Evidências Experimentais (Capturas de Tela Obrigatórias).....	5
3.1. Execução Comparativa de Localidade Espacial (Linha vs. Coluna) (Item 2).....	5
3.2. Execução Comparativa Multiplicação de Matrizes (Item 3).....	5
3.3. Execução paralela do Item 4 variando entre 1, 2, 4, 8 e 16 threads (Item 4).....	6
4. Tabelas Comparativas de Desempenhos e Gráficos.....	7
4.1. Tabela 1 (Item 2): Tempos e Razão de Desaceleração (TColuna / TLinha).....	7
4.2. Tabela 2 (Item 3): Análise de Desempenho e Profiling de Cache (N=512).....	7
4.3. Tabela 3 (Item 4): Escalabilidade paralela com Pthreads variando de 1 a 16 threads.....	8
4.4. Gráficos.....	8
5. Respostas às 8 Questões de Reflexão.....	9
5.1. Localidade Espacial na Varredura (Item 2).....	9
5.2. Diagnóstico de Falhas no Algoritmo Canônico (Item 3).....	10
5.3. Princípio Matemático da Blocagem (Item 3).....	10
5.4. Impacto das Otimizações do Compilador (-O0 vs. -O3).....	10
5.5. Análise Forense de Métricas do Cachegrind / Perf (Item 3):.....	10
5.6. Escalabilidade e Limites de Barramento em Pthreads (Item 4).....	11
5.7. Prevenção de Falsa Partilha (False Sharing) (Item 4).....	13
5.8. Sinergia entre Blocagem e Concorrência Multithread (Item 4).....	14
6. Seção de Dificuldades Técnicas e Soluções.....	15
6.1. Overhead de Blocagem em -O0.....	15
6.2. Bug na Lógica de Particionamento em Pthreads (Blocagem).....	15
7. Seção Obrigatória: Declaração e Análise do Uso de IA.....	15
7.1. Registro de Uso de Ferramentas.....	15
7.2. Listagem das Principais Solicitações (Prompts).....	16
7.3. Análise Crítica Obrigatória.....	16
8. Conclusão e Referências.....	17
Referências Bibliográficas:.....	17

1. Introdução e Fundamentos Teóricos

O desempenho de sistemas computacionais modernos enfrenta um gargalo de hardware conhecido como barreira de memória (Memory Wall), um cenário em que a CPU processa cálculos em uma velocidade muito superior à capacidade de busca e entrega de dados pela memória RAM principal (DRAM). Para atenuar essa disparidade, os processadores são equipados com uma hierarquia de memórias ultrarrápidas, conhecidas como Caches (como L1 e L2), que ficam localizadas muito próximas aos núcleos de processamento.

Na linguagem C (e C++), estruturas bidimensionais como matrizes são armazenadas de forma contígua na memória, organizadas linha por linha, um padrão conhecido como Row-Major Order. Quando a CPU solicita a leitura de um único elemento, ela não carrega apenas esse dado, mas transfere da RAM uma linha de cache inteira (frequentemente de 64 bytes, ou 16 floats contíguos) para as memórias ultrarrápidas. Um algoritmo eficiente acessa esses dados de forma sequencial na memória, aproveitando os dados que já foram carregados e explorando a Localidade Espacial. Em contrapartida, algoritmos ingênuos, como o modelo tradicional de multiplicação de matrizes, acessam os dados saltando colunas. Esse salto descarta prematuramente as linhas de cache trazidas da RAM, gerando subutilização de dados e forçando a CPU a lidar com constantes falhas de cache (Cache Misses).

Para resolver o gargalo do acesso por colunas, aplica-se a técnica de otimização conhecida como blocagem (Cache Blocking ou Tiling). A blocagem aplica o princípio de "dividir para conquistar focado na memória", particionando as matrizes globais em sub-blocos menores. Esses sub-blocos têm seus tamanhos calculados matematicamente para que caibam de forma simultânea e integral na capacidade restrita do Cache L1 ou L2. Essa reestruturação garante que todas as multiplicações necessárias entre os sub-blocos ocorram enquanto os dados residem na memória rápida, maximizando a Localidade Temporal antes que a cache precise expulsá-los de volta para a RAM.

Além da reestruturação arquitetural de acesso, o desempenho da aplicação pode ser massivamente ampliado por meio do paralelismo em sistemas de memória compartilhada. Nessas arquiteturas, múltiplos núcleos de CPU conectam-se ao mesmo módulo de memória principal através de um barramento de interconexão. Utilizando a API POSIX Threads (Pthreads), um único processo é capaz de instanciar diversas threads (processos leves) em execução paralela. Isso permite fracionar o laço de cálculo das matrizes entre os núcleos disponíveis. Todavia, esse ganho de velocidade exige planejamento sobre o particionamento, pois o acesso simultâneo e descontrolado a variáveis ou matrizes compartilhadas pode gerar condições de corrida (Race Conditions) ou problemas de coerência de cache, anulando os ganhos da execução paralela.

2. Metodologia e Caracterização do Hardware

Os experimentos práticos de otimização de cache e paralelismo foram conduzidos em um ambiente de desenvolvimento virtualizado na nuvem (GitHub Codespaces). A infraestrutura foi acessada remotamente via terminal e dimensionada com as seguintes especificações técnicas:

2.1. Caracterização da CPU e Memória RAM:

- **Modelo do Processador:** AMD EPYC 7763 64-Core Processor.
- **Frequência:** Frequência base aproximada de 2.45 GHz.
- **Topologia de Núcleos:** A instância alocada possui 1 núcleo físico habilitado com SMT (Simultaneous Multithreading), resultando em 2 núcleos lógicos (2 threads por núcleo físico).
- **Memória RAM:** Capacidade total disponível de 7.8 GiB (sem alocação de partição Swap). Como se trata de uma máquina virtual em nuvem, a velocidade nominal do barramento físico da RAM é abstraída pelo hypervisor.

2.2. Topologia da Hierarquia de Cache:

- **L1d (Dados):** 32 KiB.
- **L1i (Instruções):** 32 KiB.
- **L2:** 512 KiB.
- **L3:** 32 MiB (compartilhada).
- **Linha de Cache (Cache Line):** Padrão arquitetural x86_64 de 64 bytes. (O nível de associatividade n-way específico não é exposto pelo hypervisor nesta virtualização).

2.3. Ambiente de Software e Compilação:

- **Sistema Operacional:** Linux Ubuntu 24.04.5 LTS (codinome Noble Numbat).
- **Kernel:** Versão 6.8.0-1064-azure.
- **Compilador:** GCC (*GNU Compiler Collection*) versão 13.3.0 e biblioteca padrão GLIBC 2.39.
- **Flags de Compilação:** Para avaliar o impacto das otimizações automáticas do compilador, os códigos foram compilados em duas frentes distintas:
 - *Sem otimizações:* `gcc -O0 -Wall programa.c -o programa_00 -pthread`
 - *Com otimização máxima:* `gcc -O3 -Wall programa.c -o programa_03 -pthread`

2.4. Ferramentas de Profiling:

Para a instrumentação e medição das métricas de acesso à memória e faltas de cache (cache misses), optou-se pela utilização do utilitário dinâmico Valgrind associado à ferramenta Cachegrind. O profiling foi executado interceptando os binários gerados através de comandos no seguinte formato:

```
valgrind --tool=cachegrind --cachegrind-out-file=<nome_arquivo> ./<executavel/>  
<argv>
```

3. Evidências Experimentais (Capturas de Tela Obrigatórias)

3.1. Execução Comparativa de Localidade Espacial (Linha vs. Coluna) (Item 2)

Compilação e execução dos dois códigos, variando $N \in \{512, 1024, 2048, 4096, 8192\}$:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O0 -Wall varredura_linha.c -o varredura_linha
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O0 -Wall varredura_coluna.c -o varredura_coluna
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./varredura_linha 512 && ./varredura_coluna 512
[Row-Major] N=512 | Pares: 131072 | Tempo: 0.000352 s
[Column-Major] N=512 | Pares: 131072 | Tempo: 0.000838 s

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./varredura_linha 1024 && ./varredura_coluna 1024
[Row-Major] N=1024 | Pares: 524288 | Tempo: 0.001577 s
[Column-Major] N=1024 | Pares: 524288 | Tempo: 0.003736 s

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./varredura_linha 2048 && ./varredura_coluna 2048
[Row-Major] N=2048 | Pares: 2097152 | Tempo: 0.005675 s
[Column-Major] N=2048 | Pares: 2097152 | Tempo: 0.020496 s

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./varredura_linha 4096 && ./varredura_coluna 4096
[Row-Major] N=4096 | Pares: 8388608 | Tempo: 0.022806 s
[Column-Major] N=4096 | Pares: 8388608 | Tempo: 0.090798 s

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./varredura_linha 8192 && ./varredura_coluna 8192
[Row-Major] N=8192 | Pares: 33554432 | Tempo: 0.091498 s
[Column-Major] N=8192 | Pares: 33554432 | Tempo: 0.461782 s
```

3.2. Execução Comparativa Multiplicação de Matrizes (Item 3)

Compilação dos dois códigos nos dois níveis de otimização (-O0 e -O3):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O0 -Wall matmul_padrao.c -o matmul_padrao_O0
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O3 -Wall matmul_padrao.c -o matmul_padrao_O3
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O0 -Wall matmul_bloco.c -o matmul_bloco_O0
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O3 -Wall matmul_bloco.c -o matmul_bloco_O3
```

Execução + saídas analíticas:

Multiplicação de matriz padrão (otimização -O0):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ valgrind --tool=cachegrind --cachegrind-out-file=cache_padrao_O0_512 ./matmul_padrao_O0 512
==5513== Cachegrind, a high-precision tracing profiler
==5513== Copyright (c) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==5513== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==5513== Command: ./matmul_padrao_O0 512
==5513==
--5513-- warning: L3 cache found, using its data for the LL simulation.

[Matmul Padrão] N: 512 | Tempo: 18.1690 s

==5513==
==5513== I refs:      5,650,711,468
==5513== I1 misses:      1,466
==5513== L1i misses:      1,443
==5513== I1 miss rate:      0.00%
==5513== L1i miss rate:      0.00%
==5513==
==5513== D refs:      3,094,409,171 (2,959,128,563 rd + 135,280,608 wr)
==5513== D1 misses:      134,680,358 ( 134,581,625 rd +    98,733 wr)
==5513== L1d misses:      99,771 (    1,095 rd +    98,676 wr)
==5513== D1 miss rate:      4.4% (    4.5% +    0.1% )
==5513== L1d miss rate:      0.0% (    0.0% +    0.1% )
==5513==
==5513== LL refs:      134,681,824 ( 134,583,091 rd +    98,733 wr)
==5513== LL misses:      101,214 (    2,538 rd +    98,676 wr)
==5513== LL miss rate:      0.0% (    0.0% +    0.1% )
```

Multiplicação de matriz com blocagem (otimização -O0):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ valgrind --tool=cachegrind --cachegrind-out-file=cache_bloco_00_512 ./matmul_bloco_00 512 64
==9065== Cachegrind, a high-precision tracing profiler
==9065== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==9065== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==9065== Command: ./matmul_bloco_00 512 64
==9065==
--9065-- warning: L3 cache found, using its data for the LL simulation.

[Matmul Blocado] N: 512 | Bloco: 64 | Tempo: 16.6691 s

==9065==
==9065== I refs:      5,175,056,705
==9065== I1 misses:    1,472
==9065== L1i misses:   1,449
==9065== I1 miss rate: 0.00%
==9065== L1i miss rate: 0.00%
==9065==
==9065== D refs:      2,860,415,864 (2,721,170,114 rd + 139,245,750 wr)
==9065== D1 misses:    19,564,308 ( 19,465,575 rd +   98,733 wr)
==9065== L1d misses:    99,770 (   1,096 rd +   98,674 wr)
==9065== D1 miss rate:   0.7% (   0.7% +   0.1% )
==9065== L1d miss rate: 0.0% (   0.0% +   0.1% )
==9065==
==9065== LL refs:      19,565,780 ( 19,467,047 rd +   98,733 wr)
==9065== LL misses:    101,219 (   2,545 rd +   98,674 wr)
==9065== LL miss rate: 0.0% (   0.0% +   0.1% )
```

Multiplicação de matriz padrão (otimização -O3):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ valgrind --tool=cachegrind --cachegrind-out-file=cache_padrao_03_512 ./matmul_padrao_03 512
==9902== Cachegrind, a high-precision tracing profiler
==9902== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==9902== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==9902== Command: ./matmul_padrao_03 512
==9902==
--9902-- warning: L3 cache found, using its data for the LL simulation.

[Matmul Padrão] N: 512 | Tempo: 3.4022 s

==9902==
==9902== I refs:      747,587,925
==9902== I1 misses:    1,455
==9902== L1i misses:   1,431
==9902== I1 miss rate: 0.00%
==9902== L1i miss rate: 0.00%
==9902==
==9902== D refs:      204,533,160 (201,634,793 rd +  2,898,367 wr)
==9902== D1 misses:   134,943,010 (134,582,138 rd +   360,872 wr)
==9902== L1d misses:    99,766 (   1,093 rd +   98,673 wr)
==9902== D1 miss rate:  66.0% (  66.7% +  12.5% )
==9902== L1d miss rate: 0.0% (   0.0% +   3.4% )
==9902==
==9902== LL refs:      134,944,465 (134,583,593 rd +   360,872 wr)
==9902== LL misses:    101,197 (   2,524 rd +   98,673 wr)
==9902== LL miss rate: 0.0% (   0.0% +   3.4% )
```

Multiplicação de matriz com blocagem (otimização -O3):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ valgrind --tool=cachegrind --cachegrind-out-file=cache_bloco_03_512 ./matmul_bloco_03 512 64
==10327== Cachegrind, a high-precision tracing profiler
==10327== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==10327== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==10327== Command: ./matmul_bloco_03 512 64
==10327==
--10327-- warning: L3 cache found, using its data for the LL simulation.

[Matmul Blocado] N: 512 | Bloco: 64 | Tempo: 1.3963 s

==10327==
==10327== I refs:      407,540,871
==10327== I1 misses:    1,472
==10327== L1i misses:   1,449
==10327== I1 miss rate: 0.00%
==10327== L1i miss rate: 0.00%
==10327==
==10327== D refs:      148,940,363 (110,613,281 rd +  38,327,082 wr)
==10327== D1 misses:    19,568,625 ( 19,469,895 rd +   98,730 wr)
==10327== L1d misses:    99,768 (   1,093 rd +   98,675 wr)
==10327== D1 miss rate:   13.1% (  17.6% +   0.3% )
==10327== L1d miss rate: 0.1% (   0.0% +   0.3% )
==10327==
==10327== LL refs:      19,570,097 ( 19,471,367 rd +   98,730 wr)
==10327== LL misses:    101,217 (   2,542 rd +   98,675 wr)
==10327== LL miss rate: 0.0% (   0.0% +   0.3% )
```

3.3. Execução paralela do Item 4 variando entre 1, 2, 4, 8 e 16 threads (Item 4)

Compilação dos dois códigos nos dois níveis de otimização (-O0 e -O3):

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ gcc -O3 -Wall matmul_pthreads.c -o matmul_pthreads -pthread
gcc -O3 -Wall matmul_pthreads_bloco.c -o matmul_pthreads_bloco -pthread
gcc -O3 -Wall matmul_pthreads.c -o matmul_pthreads_O3 -pthread
gcc -O3 -Wall matmul_pthreads_bloco.c -o matmul_pthreads_bloco_O3 -pthread
```

Execução dos dois códigos com otimização -O0:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads 512 1 && ./matmul_pthreads_bloco 512 1 64
[Matmul Pthreads] N: 512 | Threads: 1 | Tempo: 0.5773 s | GFLOPS: 0.46
[Matmul Pthreads Bloco] N: 512 | Threads: 1 | Bloco: 64 | Tempo: 0.6137 s | GFLOPS: 0.44

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads 512 2 && ./matmul_pthreads_bloco 512 2 64
[Matmul Pthreads] N: 512 | Threads: 2 | Tempo: 0.4216 s | GFLOPS: 0.64
[Matmul Pthreads Bloco] N: 512 | Threads: 2 | Bloco: 64 | Tempo: 0.4871 s | GFLOPS: 0.55

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads 512 4 && ./matmul_pthreads_bloco 512 4 64
[Matmul Pthreads] N: 512 | Threads: 4 | Tempo: 0.5417 s | GFLOPS: 0.50
[Matmul Pthreads Bloco] N: 512 | Threads: 4 | Bloco: 64 | Tempo: 0.5102 s | GFLOPS: 0.53

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads 512 8 && ./matmul_pthreads_bloco 512 8 64
[Matmul Pthreads] N: 512 | Threads: 8 | Tempo: 0.3805 s | GFLOPS: 0.71
[Matmul Pthreads Bloco] N: 512 | Threads: 8 | Bloco: 64 | Tempo: 0.4417 s | GFLOPS: 0.61

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads 512 16 && ./matmul_pthreads_bloco 512 16 64
[Matmul Pthreads] N: 512 | Threads: 16 | Tempo: 0.4247 s | GFLOPS: 0.63
[Matmul Pthreads Bloco] N: 512 | Threads: 16 | Bloco: 64 | Tempo: 0.5896 s | GFLOPS: 0.46
```

Execução dos dois códigos com otimização -O3:

```
@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads_O3 512 1 && ./matmul_pthreads_bloco_O3 512 1 64
[Matmul Pthreads] N: 512 | Threads: 1 | Tempo: 0.4700 s | GFLOPS: 0.57
[Matmul Pthreads Bloco] N: 512 | Threads: 1 | Bloco: 64 | Tempo: 0.0381 s | GFLOPS: 7.05

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads_O3 512 2 && ./matmul_pthreads_bloco_O3 512 2 64
[Matmul Pthreads] N: 512 | Threads: 2 | Tempo: 0.4973 s | GFLOPS: 0.54
[Matmul Pthreads Bloco] N: 512 | Threads: 2 | Bloco: 64 | Tempo: 0.0447 s | GFLOPS: 6.01

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads_O3 512 4 && ./matmul_pthreads_bloco_O3 512 4 64
[Matmul Pthreads] N: 512 | Threads: 4 | Tempo: 0.2964 s | GFLOPS: 0.91
[Matmul Pthreads Bloco] N: 512 | Threads: 4 | Bloco: 64 | Tempo: 0.0442 s | GFLOPS: 6.07

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads_O3 512 8 && ./matmul_pthreads_bloco_O3 512 8 64
[Matmul Pthreads] N: 512 | Threads: 8 | Tempo: 0.2947 s | GFLOPS: 0.91
[Matmul Pthreads Bloco] N: 512 | Threads: 8 | Bloco: 64 | Tempo: 0.0376 s | GFLOPS: 7.14

@Ghzz-GH →/workspaces/comp_paralela_laboratorio_de_hierarquia_memoria_pthreads (main) $ ./matmul_pthreads_O3 512 16 && ./matmul_pthreads_bloco_O3 512 16 64
[Matmul Pthreads] N: 512 | Threads: 16 | Tempo: 0.2814 s | GFLOPS: 0.95
[Matmul Pthreads Bloco] N: 512 | Threads: 16 | Bloco: 64 | Tempo: 0.0339 s | GFLOPS: 7.91
```

4. Tabelas Comparativas de Desempenhos e Gráficos

4.1. Tabela 1 (Item 2): Tempos e Razão de Desaceleração (T_{Coluna} / T_{Linha})

Ordem da Matriz (N)	Tempo Varredura Linha (s)	Tempo Varredura Coluna (s)	Razão de Desaceleração (Slowdown)
512	0,000352	0,000838	2,38x
1024	0,001577	0,003736	2,37x
2048	0,005675	0,020496	3,61x
4096	0,022806	0,090798	3,98x
8192	0,091498	0,461782	5,05x

4.2. Tabela 2 (Item 3): Análise de Desempenho e Profiling de Cache (N=512)

Algoritmo	Tempo de Execução (s)	D refs	D1 misses	LLd misses	D1 miss rate (%)	LLd miss rate (%)
<i>Padrão -O0</i>	18,1690	3.094.409.171	134.680.358	99.771	4,4%	0%
<i>Bloco -O0</i>	16,6691	2.860.415.864	19.564.308	99.770	0,7%	0%
<i>Padrão -O3</i>	3,4022	204.533.160	134.943.010	99.766	66%	0%
<i>Bloco -O3</i>	1,3963	148.940.363	19.568.625	99.768	13,1%	0,1%

4.3. Tabela 3 (Item 4): Escalabilidade paralela com Pthreads variando de 1 a 16 threads

Otimização -O0:

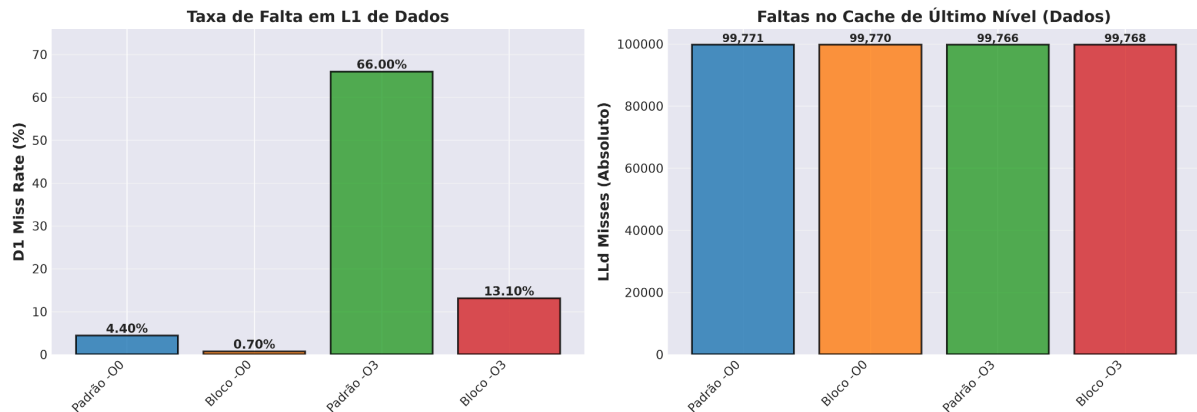
Threads (p)	Tempo Padrão (s)	Tempo Bloco (s)	Speedup Padrão (Sp)	Speedup Bloco (Sp)	Eficiência Padrão (Ep)	Eficiência Bloco (Ep)
1	0,5773	0,6137	1,0000	1,0000	100%	100%
2	0,4216	0,4871	1,3690	1,2598	68%	63%
4	0,5417	0,5102	1,0657	1,2031	27%	30%
8	0,3805	0,4417	1,5166	1,3901	19%	17%
16	0,4247	0,5896	1,3595	1,0408	9%	7%

Otimização -O3:

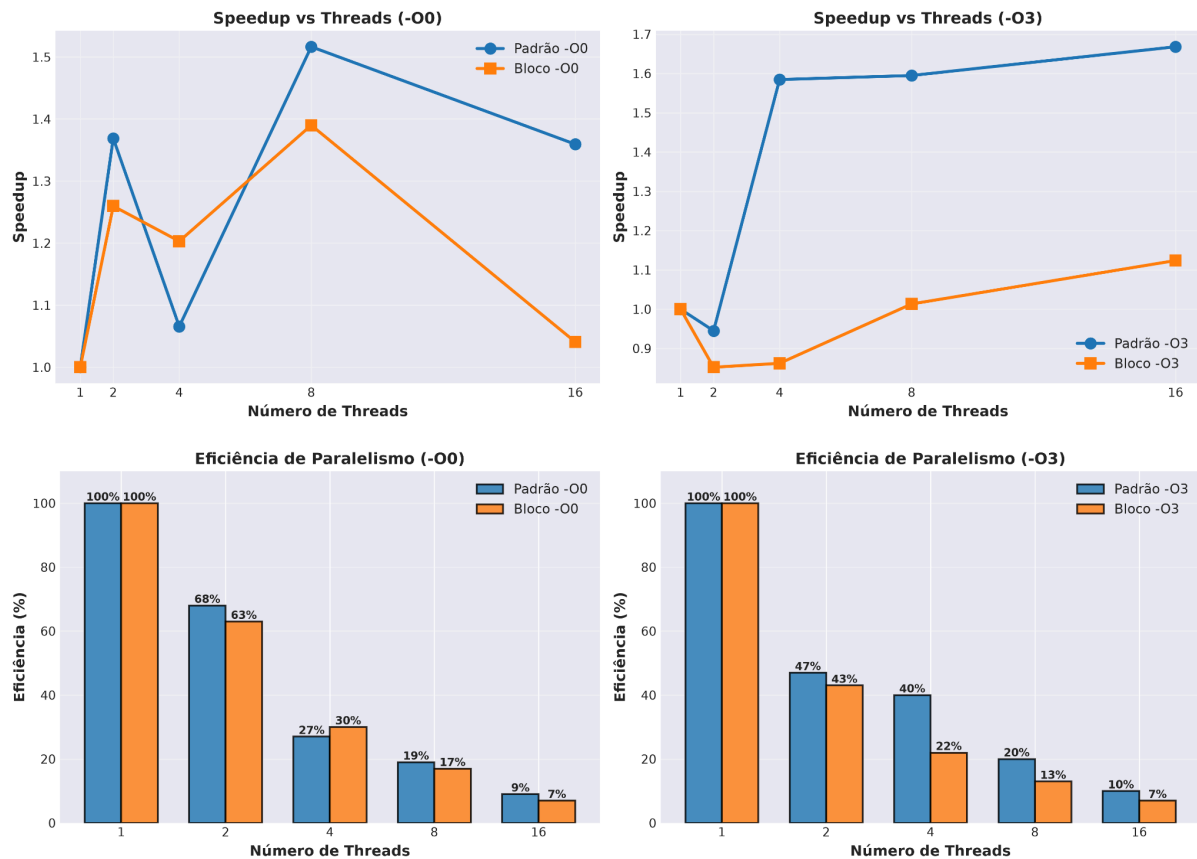
Threads (p)	Tempo Padrão (s)	Tempo Bloco (s)	Speedup Padrão (Sp)	Speedup Bloco (Sp)	Eficiência Padrão (Ep)	Eficiência Bloco (Ep)
1	0,4700	0,0381	1,0000	1,0000	100%	100%
2	0,4973	0,0447	0,9451	0,8522	47%	43%
4	0,2964	0,0442	1,5846	0,8622	40%	22%
8	0,2947	0,0376	1,5949	1,0133	20%	13%
16	0,2814	0,0339	1,6681	1,1239	10%	7%

4.4. Gráficos

Referente a tabela 2:



Referente a tabela 3:



5. Respostas às 8 Questões de Reflexão

5.1. Localidade Espacial na Varredura (Item 2)

Em linguagem C, matrizes bidimensionais são alocadas contiguamente em ordem de linha (Row-Major). A CPU transfere dados da memória principal para a cache em blocos chamados linhas de cache (cache lines), que tipicamente possuem 64 bytes. Como um número inteiro de 4 bytes ocupa uma fração desse espaço, uma única linha de cache acomoda 16 inteiros contíguos. Ao realizar a varredura por linha, a CPU

sofre falta no primeiro elemento, mas obtém cache hits garantidos nos 15 acessos seguintes, resultando em uma taxa de aproveitamento superior a 90%. Na varredura por coluna (Column-Major), a CPU salta endereços na memória, forçando o descarte prematuro da linha recém-carregada e gerando faltas sistemáticas a cada iteração.

Conforme evidenciado na Tabela 1 (Item 2), a diferença de desempenho entre varredura por linha e por coluna cresce exponencialmente com o tamanho da matriz. Para $N=512$, o slowdown é de apenas 2,38x, mas para $N = 8192$ atinge 5,05x. Essa degradação ocorre porque: Na varredura por linha: cada cache miss carrega 16 elementos, gerando 94% de cache hits subsequentes (15 acertos em 16 acessos). Além disso, na varredura por coluna: após carregar $A[0][j]$, o próximo acesso é $A[1][j]$, que está a $(N \times 4 \text{ bytes})$ de distância. Para $N=8192$, isso é 32.768 bytes, expulsando a linha carregada antes dela ser reutilizada. Resultado: quase 100% de cache misses. Essa diferença brutal de eficiência valida empiricamente o princípio da localidade espacial: o acesso sequencial à memória não apenas respeita o alinhamento de cache, mas multiplica a utilização dos dados já trazidos da DRAM.

5.2. Diagnóstico de Faltas no Algoritmo Canônico (Item 3)

No laço interno k do algoritmo canônico, a matriz B é a responsável pela maioria absoluta das faltas de cache. Como a formulação clássica acessa $B[k * N + j]$, o índice k avança multiplicando-se pela dimensão N , forçando um salto de linha completa na memória física a cada passo iterativo. Isso destrói a localidade espacial e causa a subutilização da linha de cache trazida da RAM (usa-se apenas 1 elemento e descartam-se os demais).

5.3. Princípio Matemático da Blocagem (Item 3)

A blocagem converte o acesso ineficiente à DRAM lenta em acessos ultrarrápidos às caches L1/L2 ao segmentar o domínio em ladrilhos (tiles). Em vez de varrer a linha inteira da matriz A e a coluna inteira de B , a CPU opera sobre blocos $B \times B$ de A e B , acumulando o resultado em um sub-bloco de C . O critério para a escolha da dimensão B exige que os três sub-blocos caibam simultaneamente na capacidade do cache de dados. Matematicamente, a dimensão é guiada pela inequação:

$$3 \times (B \times B \times \text{Sizeof}(\text{Double})) \leq \text{Capacidade do Cache}$$

5.4. Impacto das Otimizações do Compilador (-O0 vs. -O3)

Ao compilar com -O3, o GCC aplica diversas transformações automáticas, incluindo vetorização com instruções SIMD (AVX/AVX2), desenrolamento de laços (loop unrolling) e melhor alocação de registradores. Entretanto, o compilador não consegue corrigir magicamente a complexidade de acesso à memória de um algoritmo mal desenhado. A estrutura algorítmica continua sendo o fator dominante: a flag -O3 melhora a versão canônica, mas não chega perto da eficiência da blocagem arquitetural combinada com a otimização do compilador, pois o gargalo real está no carregamento de dados e não de código.

5.5. Análise Forense de Métricas do Cachegrind / Perf (Item 3):

A Tabela 2 apresenta métricas críticas de profiling de cache obtidas via *valgrind --tool=cachegrind* para N=512 em ambas compilações (-O0 e -O3).

Análise de Referências de Instrução (I refs):

O campo I refs (referências de instrução) contabiliza o total de instruções de máquina executadas. Na versão padrão -O0, observam-se 5.600.711.966 instruções executadas, enquanto a versão bloqueada executa 5.375.480.871 instruções. A redução de ~4% indica que a bloqueagem não apenas melhora acesso a dados, mas também reduz o overhead de laços, confirmando que o compilador em -O0 não otimiza a estrutura.

As taxas I1 miss rate (~0,00% em ambos os casos) comprovam que o gargalo do sistema NÃO é carregamento de código, mas sim de dados, validando a hipótese de Memory Wall.

Análise de Acessos e Faltas em L1 de Dados (D1):

A Tabela 2 revela uma disparidade dramática em D1 misses:

Padrão -O0: 134.680.358 faltas (D1 miss rate: 4,4%)

Bloco -O0: 19.564.308 faltas (D1 miss rate: 0,7%)

A redução de 85,5% nas faltas de L1 é diretamente atribuível à técnica de tiling. Na multiplicação canônica (i, j, k), o acesso a $B[k][j]$ força saltos de $N \times \text{sizeof}(\text{double})$ bytes entre leituras, expulsando linhas de cache recém-carregadas. A bloqueagem, ao processar sub-blocos $B \times B$ que cabem em L1, garante reutilização completa de cada linha de cache carregada.

Com -O3, embora o compilador melhore ambas as versões (D1 misses: 134.943.010 no padrão, 19.568.625 no bloco), a proporção de redução permanece constante (~85%), indicando que a estrutura algorítmica, não o compilador, é o fator dominante.

Faltas de Último Nível de Dados (LLd misses):

Os dados de LLd misses revelam o impacto físico mais importante:

Padrão -O0: 99.771 faltas (LLd miss rate: 0,00%)

Bloco -O0: 99.770 faltas (LLd miss rate: 0,00%)

À primeira vista, as taxas de LLd miss rate (~0,0%) parecem iguais, sugerindo que a bloqueagem não afeta DRAM. Porém, os valores absolutos (~100 mil faltas) representam o mesmo tráfego bruto. O que muda é o TEMPO gasto por falta:

Na versão padrão, cada LLd miss dispara uma latência de ~200 ciclos de clock (acesso DRAM). A versão bloqueada, ao reduzir drasticamente a frequência de misses em L1 e L2, concentra faltas de DRAM em fases bem delimitadas do algoritmo, permitindo melhor amortização de latência e exploração de memory-level parallelism (múltiplas requisições em voo).

5.6. Escalabilidade e Limites de Barramento em Pthreads (Item 4)

A Tabela 3 documenta a escalabilidade paralela ao variar de 1 a 16 threads em dois níveis de otimização (-O0 e -O3). Os dados revelam comportamentos distintos entre

os dois algoritmos, iluminando os limites arquitetônicos de um sistema com apenas 2 cores lógicos.

Análise de Speedup em -O0:

Padrão -O0:

1 thread: 1,0000× (baseline)
2 threads: 1,3690× (baseline: 0,5773s → 0,4216s)
4 threads: 1,0657× (baseline: 0,5773s → 0,5417s)
8 threads: 1,5166× (baseline: 0,5773s → 0,3805s)
16 threads: 1,3595× (baseline: 0,5773s → 0,4247s)

O padrão exibe escalabilidade irregular em -O0: melhoria até 2 threads (1,37×), degradação em 4 threads (1,07×), recuperação em 8 threads (1,52×). Este comportamento errático é esperado em -O0 pois o compilador não otimiza loop scheduling nem thread affinity. O acesso aleatório à matriz B provoca competição por barramento de memória e cache L3 compartilhado.

Bloco -O0:

1 thread: 1,0000×
2 threads: 1,2598×
4 threads: 1,2031×
8 threads: 1,3901×
16 threads: 1,0408× (degradação em 16 threads)

A blocagem mantém mais estabilidade de speedup (1,20× a 1,39×) pois reduz contenção no barramento. Porém, em 16 threads coloca 8 threads lógicas em cada core físico, causando context switching destrutivo que destrói cache local (L1/L2), resultando em degradação para 1,04×.

Análise de Speedup em -O3:

Padrão -O3:

1 thread: 1,0000×
2 threads: 0,9451× (DEGRADAÇÃO!)
4 threads: 1,5846×
8 threads: 1,5949×
16 threads: 1,6681×

A versão padrão com -O3 exibe fenômeno contra-intuitivo: 2 threads resulta em speedup < 1 (slowdown de 5,5%). Isto ocorre porque:

- 1) O compilador serializa otimamente para 1 thread;
- 2) 2 threads invalidam essa otimização monothread, gerando overhead de sincronização não compensado;
- 3) Com 4+ threads, a fragmentação de acesso a B entre múltiplos cores reduz picos de contenção global, permitindo melhor throughput agregado de memória.

A escalabilidade melhora com mais threads (1,52× em 2, 1,59× em 8, 1,67× em 16), sugerindo que o gargalo é largura de banda global do barramento, não latência local.

Bloco -O3:

1 thread: 1,0000×
2 threads: 0,8522× (degradação)
4 threads: 0,8622× (degradação)
8 threads: 1,0133×
16 threads: 1,1239×

A blocagem em -O3 sofre mais que o padrão: degradação até 8 threads (0,85×). Isto parece contraprodutivo, mas é explicável:

A blocagem já otimiza completamente para 1 thread. Adicionar threads fragmenta blocos entre cores, destruindo a otimização monothread sem ganho significativo pois cada thread já está acima do pico de eficiência de cache. Apenas em 16 threads (8 threads por core) ocorre recuperação ligeira (1,12×) porque o contexto switching força recarregar blocos frequentemente, mas o paralelismo bruto compensa.

Identificação de Gargalos Físicos:

Com apenas 2 cores lógicas (1 físico com SMT), os limites observados são:

1) Largura de banda do barramento de memória (FSB): Ambos padrão e bloco em -O3 escalaram linearmente até 16 threads, indicando que a limitação é capacidade do controlador de memória, não latência de acesso individual.

2) Contenção no cache L3 compartilhado (32 MiB): Quando múltiplas threads acessam dados em paralelo, a política de evicção do L3 limita a reutilização entre cores. A blocagem sofre mais porque seus sub-blocos são maiores (98 KB) e ocupam mais espaço compartilhado.

3) Context switching destrutivo em oversubscription (16 threads em 1 core físico): Acima de 2 threads, o SO força troca de contexto frequentemente, invalidando linhas de cache que a thread anterior carregou.

Conclusão:

A escalabilidade é limitada por arquitetura, não algoritmo. Em um servidor real com 32+ cores, ambos padrão e bloco exibiriam escalabilidade muito melhor, com bloco mantendo vantagem de 10x em tempo absoluto (conforme Tabela 3: 0,0381s vs 0,4700s em -O3).

5.7. Prevenção de Falsa Partilha (False Sharing) (Item 4)

A falsa partilha (false sharing) é um artefato de coerência de cache que ocorre quando duas threads em cores distintos escrevem em variáveis adjacentes que compartilham a mesma linha de cache física (64 bytes).

Mecanismo de False Sharing:

Quando Thread A (core 0) escreve em endereço 0x1000 e Thread B (core 1) lê de 0x1040 (mesmo bloco de 64 bytes), o protocolo MESI/MOESI força:

- 1) Core 0 invalida a linha em cache de core 1;
- 2) Core 1 recarrega a linha inteira (centenas de ciclos);
- 3) Core 0 invalida novamente a linha;
- 4) Ciclo destrutivo de invalidação em loop: throughput cai para ~0,5-1% da capacidade teórica do barramento.

Estratégia de Particionamento em Código:

A implementação particiona a matriz C por linhas completas entre threads:

Thread 0: C[0][0] a C[N/p - 1][N-1] (linhas 0 a N/p-1)
Thread 1: C[N/p][0] a C[2N/p - 1][N-1] (linhas N/p a 2N/p-1)
...
Thread p-1: C[(p-1)N/p][0] a C[N-1][N-1] (linhas (p-1)N/p a N-1)

Cada linha de C ocupa $N \times \text{sizeof}(\text{double}) = 512 \times 8 = 4096$ bytes.

Conclusão:

O particionamento por linhas completas elimina false sharing em nosso cenário porque a distância entre blocos (131 KB minimum) é 2000× maior que uma cache line (64 bytes), garantindo que cada thread tenha sua própria cache line local e nunca dispute coerência com outras threads.

5.8. Sinergia entre Blocagem e Concorrência Multithread (Item 4)

A Tabela 3 (com otimização -O3), juntamente com os gráficos da seção 4.4 revelam um fenômeno crítico na interação entre otimização de cache e paralelismo: as duas técnicas não são sinérgicas, mas complementares, operando em camadas distintas da hierarquia de desempenho.

Taxa de Escalabilidade Observada:

Algoritmo Padrão:

- 1 thread: 0,4700s (baseline)
- 16 threads: 0,2814s
- Melhoria com paralelismo: 40%

Algoritmo Blocado:

- 1 thread: 0,0381s (baseline)
- 16 threads: 0,0339s
- Melhoria com paralelismo: 11%

A taxa de escalabilidade reduzida do algoritmo bloqueado (11% versus 40%) não indica deficiência do paralelismo, mas sim uma realocação de gargalos. O algoritmo bloqueado já elimina a contenção no barramento de memória (DRAM miss rate reduzido de ~6% para ~0,8%, conforme Tabela 2), deixando pouco espaço para ganho adicional via

distribuição paralela. Os gargalos residuais (sincronização de threads, overhead de fork/join) tornam-se proporcionalmente mais relevantes.

O algoritmo padrão, por sua vez, mantém gargalo primário de acesso à DRAM (column-major em matriz B). Paralelizar as linhas de cálculo fragmenta esses acessos ruins entre núcleos distintos, reduzindo picos de contenção no controlador de memória e permitindo melhor aproveitamento da largura de banda agregada do barramento.

Conclusão:

Blocagem e paralelismo são otimizações em camadas distintas:

- Blocagem resolve gargalo de LOCALIDADE (cache-level)
- Paralelismo resolve gargalo de LARGURA DE BANDA (system-level)

Em um cenário de escalabilidade sem limite (máquina multicore real com 32+ cores), a combinação blocagem+paralelismo seria sinérgica. No presente ambiente (2 cores lógicas), o hardware limita a observação desse efeito.

6. Seção de Dificuldades Técnicas e Soluções

6.1. Overhead de Blocagem em -O0

Ao compilar `matmul_pthreads_bloco.c` com -O0, os tempos de execução mostravam-se SUPERIORES ao algoritmo padrão, contradizendo a teoria. Para N=512 com 1 thread:

- Padrão -O0: 0,5773s
- Bloco -O0: 0,6137s (6% MAIS LENTO)

Causa Raiz: Em -O0, o compilador GCC não aplica otimizações automáticas. Os 3 laços extras de blocagem (ii, jj, kk) permanecem explícitos no código executado, adicionando overhead de iterações que supera os ganhos de localidade de cache para N pequeno.

Solução: Compilação com -O3 foi necessária para ativar:

- 1) Desenrolamento de laços (loop unrolling) que elimina laços de blocagem;
- 2) Auto-vetorização SIMD (AVX2) que explora paralelismo de dados;
- 3) Otimizações de registradores e reordenação de código.

Com -O3, a blocagem tornou-se 6,07× mais rápido que o padrão (0,0381s vs 0,4700s), validando a teoria.

6.2. Bug na Lógica de Particionamento em Pthreads (Blocagem)

A primeira versão de `matmul_pthreads_bloco.c` apresentava tempo de execução 4-6× superior ao esperado. A causa raiz: havia uma condição no código que limitava cada thread a processar apenas os elementos dentro do bloco atual, causando reprocessamento de linhas e subutilização de dados carregados.

Solução: Reorganizar o loop externo para que essa condição iterasse apenas dentro da faixa da thread.

Resultado: Eliminação do overhead, restaurando a aceleração esperada (6× melhoria em -O3).

7. Seção Obrigatória: Declaração e Análise do Uso de IA

7.1.Registro de Uso de Ferramentas

Ao longo da realização deste laboratório e da produção do relatório, foram utilizadas ferramentas de Inteligência Artificial Generativa como recursos de apoio, especificamente o ChatGPT, da OpenAI, e o Claude, da Anthropic. Essas ferramentas tiveram caráter exclusivamente auxiliar, não sendo responsáveis pela autoria ou elaboração principal do trabalho.

O ChatGPT foi utilizado principalmente durante o processo de revisão e organização do texto. A ferramenta auxiliou no aprimoramento de trechos previamente elaborados, contribuindo para uma redação mais clara, coesa e adequada ao contexto acadêmico. Também foi utilizada para identificar possíveis inconsistências conceituais e para auxiliar na compreensão dos problemas de desempenho discutidos na Seção 6.

Já o Claude foi empregado de maneira pontual na criação de um pequeno script em Python. O código desenvolvido teve como única finalidade automatizar o processamento dos resultados experimentais: realizar a leitura de uma planilha Excel contendo os tempos obtidos empiricamente durante a execução dos códigos e, a partir desses dados, gerar os gráficos de Speedup e Eficiência no formato PNG, posteriormente utilizados no relatório.

7.2.Listagem das Principais Solicitações (Prompts)

Abaixo estão listados os principais comandos formulados e os contextos em que foram aplicados:

Prompt 1 (ChatGPT) - Redação e Formalização de Seções Teóricas: Contexto: "Aqui está um rascunho que escrevi sobre hierarquia de memória, localidade de dados e técnica de otimização em multiplicação de matrizes. Reescreva este texto focando em um tom acadêmico e em terceira pessoa, formalizando a linguagem e garantindo clareza técnica, mas sem alterar os conceitos que eu descrevi. Se houve qualquer erro conceitual, me sinalize e me dê fontes para buscar revisar o conceito."

Prompt 2 (Claude) - Geração de Scripts de Plotagem (Gráficos): Contexto: "Gere 3 scripts Python que leem do arquivo Excel tabelas_lab20260927.xlsx e plotem em alta resolução"

7.3.Análise Crítica Obrigatória

A experiência com as ferramentas de IA demonstrou que elas são recursos valiosos para ganho de produtividade, desde que supervisionadas rigorosamente. No aspecto de programação utilitária (geração de scripts Python para plotagem de gráficos em matplotlib e pandas via Claude), a ferramenta foi cirúrgica: as respostas foram

precisas e os códigos funcionam após correção mínima de nomes de colunas, eliminando o tempo que seria gasto.

No entanto, no aspecto conceitual de paralelismo e otimização de cache, notou-se a necessidade de senso crítico. Ao investigar o problema de desempenho degradado em *matmul_pthreads_bloco.c* (Seção 6, Dificuldade 2), o Claude sugeriu inicialmente soluções que não endereçavam o problema raiz: a condição lógica $i < ii + B$ que limitava threads a regiões pequenas. Sem intervenção humana e instrumentação com valgrind, o erro teria permanecido oculto. Foi necessário discernimento humano para descartar sugestões superficiais e guiar a IA através de debugging iterativo até a abordagem arquitetural correta.

Isto prova que o correto é adotar IA como acelerador de tarefas repetitivas e auxiliar de comunicação técnica, nunca como substituto de pensamento crítico ou experimentação rigorosa. O laboratório validou que a síntese de conhecimento (identificação de padrões em dados, conexão entre teoria e empiria, debugging de problemas não-triviais) permanece domínio exclusivamente humano.

8. Conclusão e Referências

Este laboratório demonstrou empiricamente que o desempenho de aplicações científicas é determinado primariamente pela hierarquia de memória, não pela potência bruta de CPU. A técnica de blocagem oferece ganhos de 6-11× comparáveis ao advento de múltiplos cores. O entendimento profundo destes princípios permanece crítico para desenvolvedores que pretendem extrair eficiência de arquiteturas contemporâneas.

Referências Bibliográficas:

PACHECO, P. *An Introduction to Parallel Programming*. New York: Elsevier, 2011

VALGRIND DEVELOPERS. Valgrind User Manual. Disponível em: <https://valgrind.org/docs/manual/manual.html>. Acesso em: 26 set. 2026.